Day 5 — LlamaIndex, LangChain, MCP, and framework selection

Outcome

Be able to place each tool in the correct architectural category, use it behind internal interfaces, and justify both adoption and non-adoption.

1. Category map

Item	Category	Primary question
Vanilla RAG	Design pattern	How do I retrieve evidence before generation?
LlamaIndex	Data/context framework	How do I ingest, index, retrieve, and synthesize over private data?
LangChain	LLM application framework	How do I compose models, prompts, retrievers, tools, output, and middleware?
LangGraph	Stateful orchestration runtime	How do I control branches, loops, persistence, approval, and recovery?
MCP	Connectivity protocol	How do AI hosts discover and communicate with external tools, resources, and prompts?

None of these is the LLM, vector database, identity system, policy engine, complete application platform, or automatic correctness layer.

2. LlamaIndex

LlamaIndex implements context augmentation: it connects application questions to private or current data so the model receives relevant runtime context rather than relying only on pretrained knowledge.

Building blocks

- **Document:** source-level unit plus metadata.
- **Node:** smaller searchable unit derived from a document.
- **Chunk:** text span; commonly represented as a node.
- **Loader/connector:** obtains source data.
- **Parser:** extracts and structures content.
- **Ingestion pipeline:** cleaning, metadata, chunking, embedding, and indexing.
- **Index:** organized access structure.
- **Retriever:** returns evidence nodes.
- **Query engine:** coordinates retrieval and response.
- **Postprocessor:** filters, transforms, or reranks nodes.
- **Response synthesizer:** produces an answer from selected evidence.

Index families

- Vector store: semantic search.
- Summary: broad synthesis.
- Tree/hierarchical: broad-to-detailed navigation.
- Keyword table: exact terms.
- Property graph: entities and relationships.

Vector retrieval is a common starting point, not a universal final design.

Retriever versus query engine

```
retriever: question → evidence
query engine: question → retrieval → synthesis → result
```

Keep retrieval available as a lower-level boundary when the application owns prompting, validation, or orchestration elsewhere.

Production guidance

- Make ingestion incremental, idempotent, versioned, recoverable, and testable.
- Keep identity, lifecycle, security, and citation metadata.
- Validate LLM-generated filters.
- Combine dense and sparse retrieval where exact terms matter.
- Store framework versions and trace parsing/retrieval/synthesis.
- Do not expose LlamaIndex types across domain, API, and storage layers.

Internal boundary:

```
class KnowledgeRetriever:
    def retrieve(self, question, access_scope) -> list[Evidence]:
        ...
```

A `LlamaIndexKnowledgeRetriever` translates between this contract and framework objects.

When it fits

- complex document ingestion;
- several source types;
- index/retriever/query-engine reuse;
- advanced data-aware workflows;
- response synthesis and citation needs.

It may be unnecessary for a small direct vector search and one prompt.

3. LangChain

Building blocks

- Model interfaces.
- Prompt templates and few-shot examples.
- Structured output and parsers.
- Retrievers.

- Tools.
- Chains.
- State and memory.
- Callbacks, middleware, streaming, and tracing.

Chain versus agent

Chain:

```
retrieve → prompt → model → structured answer
```

The developer fixes the order.

Agent:

```
model decides whether to call a bounded tool, observe, and continue
```

Use a chain when the workflow is known. Dynamic choice must create enough value to justify higher unpredictability, testing, latency, and security cost.

Structured output

A schema improves syntax and type reliability, but not factual or business validity.

```
schema-valid ≠ evidence-valid ≠ authorized
```

Validate evidence, allowed values, business rules, permissions, and side effects outside the model.

State versus memory

- State: data needed in the current execution.
- Memory: retained information across interactions.

Do not send unlimited history. Retrieve only relevant history to control privacy, injection surface, tokens, and latency.

Framework risks

- hidden call chains and debugging difficulty;
- provider feature mismatch behind common interfaces;
- framework/version coupling;
- unclear state ownership;
- excess abstraction for simple calls;
- weak tests and observability;
- cost/latency hidden by convenience.

Keep chains short, use explicit interfaces, validate boundaries, trace each component, and move to LangGraph when explicit workflow control is needed.

The source notes place LangSmith alongside LangChain callbacks and tracing for inspecting calls, latency, tokens, failures, and evaluation. Tool choice does not remove the need for application-owned safe telemetry and quality tests.

4. LlamaIndex versus LangChain

Need	Stronger starting point
Complex ingestion/parsing/indexing	LlamaIndex
Data-backed query engines/synthesis	LlamaIndex
Model/provider/prompt/tool composition	LangChain
Tool-calling application harness	LangChain
Explicit persisted branching workflow	LangGraph
One provider call	Direct SDK may be clearer

They can be combined:

```
LlamaIndex ingests/indexes and exposes retrieval
→ LangChain composes prompt/model/tool/output
→ LangGraph controls multi-step workflow
```

Adopt only the needed layers.

5. MCP foundations

The Model Context Protocol (MCP) standardizes connectivity, not reasoning or workflow.

```
host
  → MCP client
    → MCP server
      → business system
```

Roles

- **Host:** complete AI experience; model, UX, policy, approvals, workflow, and server trust.
- **Client:** protocol connection to one server; lifecycle, requests, responses, cancellation, and notifications.
- **Server:** exposes capabilities and translates to the real backend.

Capabilities

- **Tool:** callable operation; may read or mutate.
- **Resource:** contextual information.
- **Prompt:** discoverable reusable message template.

A prompt is not a workflow. A resource is not automatically safe. A standard tool connection is not automatic authorization.

Lifecycle and session

```
initialize
→ negotiate protocol/capabilities
→ operate
→ shutdown
```

Protocol session state tracks connection/capabilities/pending work. It is not conversation memory or durable workflow state.

JSON-RPC mental model

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "get_incident",
    "arguments": {"incident_id": "INC-17"}
  }
}
```

The response uses the same ID. Messages include requests, responses, errors, and notifications.

6. MCP versus function calling

Function calling lets a model request a structured invocation.

MCP standardizes how an application discovers and communicates with systems exposing capabilities.

```
model function call
→ host policy/validation
→ MCP client/server call
→ backend
```

MCP is not an agent framework and does not own planning, branching, retries, or workflow recovery.

MCP versus A2A

MCP connects an AI host to external tools, resources, and prompts. A2A describes agent-to-agent interoperability, where one agent communicates or collaborates with another. They solve different boundaries and can coexist in one platform.

7. MCP governance and security

Responsibility split

Host:

- trust and server selection;
- tool exposure to the model;
- approval and policy;
- whether results enter context;
- combining several servers.

Server:

- backend authentication/authorization;
- argument validation;
- safe error translation;

- secret protection;
- result structure.

Governance:

- approved catalogue;
- ownership;
- data classification;
- roles and permissions;
- approval policy;
- audit retention;
- version/deprecation standards.

Least privilege

Enforce at every layer:

```
user permissions
→ host tool filtering
→ server authorization
→ downstream credentials
→ data controls
```

Never depend on model obedience.

Narrow tools

Avoid:

```
execute_any_operation(operation, payload)
execute_sql(sql)
run_shell(command)
```

Prefer:

```
get_incident(incident_id)
create_incident(summary, customer_id)
restart_approved_service(service_name, environment)
```

Separate preparation from high-risk execution:

```
prepare change → show impact → approve → apply
```

Approval and audit

Approval should show the real target and effect, not a vague “allow tool?” prompt.

Audit:

- principal, host, model/workflow;
- server/tool/version;
- target and argument hash;
- approval;
- result/status;
- duration and errors.

Do not log full sensitive records.

Prompt injection and data leakage

External documents, tickets, web pages, tool results, and tool descriptions are untrusted data.

Controls:

- separate trusted instructions and external data;
- permission-aware tool discovery;
- deterministic policy checks;
- tool allowlists and egress restrictions;
- result validation and data classification;
- approval for writes;
- tenant isolation and redaction.

Sessions are not authentication.

8. Production MCP operations

MCP is worth adopting when several AI applications reuse the same business systems, portable discovery is valuable, platform teams need standardized connectors, or governance benefits from a common capability model.

It may be unnecessary for one small application calling one stable API or a local calculation with no expected reuse, especially where another service layer adds latency and operational burden.

Challenges:

- too many tools presented to the model;
- ambiguous schemas/descriptions;
- weak server ownership;
- inconsistent results;
- latency and dependency failures;
- incompatible versions;
- missing traces/audit.

Optimizations:

- domain routing/server selection;
- permission-aware discovery;
- semantic tool retrieval;
- concise schemas;
- only expose a small relevant set;
- timeouts and bounded retries;
- cache safe read-only data;
- validate output;
- publish server quality standards.

Retry read operations more safely than non-idempotent writes. A timeout on a write may leave the external outcome unknown; reconcile before retrying.

Framework-boundary and upgrade testing

Keep the application's tests layered:

- unit-test domain services with framework-neutral evidence, tool, and model fakes;
- contract-test each adapter's translation of framework objects and errors;
- integration-test important ingestion, retrieval, structured-output, and MCP paths;
- run golden behavioral and failure cases before changing framework, model, prompt, parser, index, or tool versions;
- pin compatible versions and keep a rollback target for production upgrades.

The purpose is not to test a framework's internals. It is to prove that an upgrade preserves the application contract, observable behavior, safety policy, and required provider features.

9. Integrated architecture

```
API/UX
  ↓
LangGraph workflow and state
  ├─ LangChain model/prompt/tool components
  ├─ LlamaIndex-backed knowledge retriever
  ├─ policy/approval service
  └─ MCP clients
       ↓
     domain MCP servers
       ↓
     databases/internal APIs/SaaS
```

Memory statement:

```
RAG retrieves.
LlamaIndex organizes data and retrieval.
LangChain composes LLM application components.
LangGraph orchestrates stateful workflows.
MCP connects external capabilities.
IAM/policy authorizes.
Business services execute.
```

Worked example: enterprise policy assistant

```
LlamaIndex:
  connect to approved policy sources
  → preserve headings/tables/metadata
  → create nodes and indexes
  → expose permission-aware retrieval

LangChain:
  compose the retriever, prompt, model, structured response,
  callbacks/middleware, and bounded read/write tools

LangGraph:
  route greeting, policy question, live lookup, proposed action,
  approval, escalation, and recovery through explicit state

MCP:
  connect approved read/write capabilities to policy, HR,
  scheduling, or case-management backends
```

Example request lifecycle:

```
authenticate and derive access scope
→ retrieve current policy evidence
→ call an approved read tool when live state is needed
→ build grounded structured response
→ validate citations and answerability
→ if a write is proposed, show target/effect and request approval
→ authorize and execute through the business service
→ validate result and record trace/audit
```

The same use case may need only direct retrieval and one prompt. Add each framework/protocol layer only when its distinct responsibility is required.

10. Selection questions

1. Is external knowledge required?
2. Is direct retrieval simple enough?
3. Are reusable models, prompts, tools, or providers needed?
4. Is the flow a single request/response?
5. Are branches, loops, persisted state, recovery, or approval required?
6. Will many applications reuse the same external capabilities?
7. Are tool actions consequential?
8. Can the team trace, test, upgrade, and operate the abstraction?

Project-grounded example: each framework had one architectural job

Project scenario. DPDK BenchOps Copilot needed to ingest fragmented benchmark knowledge, answer grounded questions, execute multi-step flows, and access operational data without allowing an LLM to construct arbitrary commands. The selected stack matched those separate needs:

Project component	Real responsibility
LlamaIndex	Ingest logs, database JSON records, tuning documents, methodology guides, historical metadata, and notes; normalize, chunk, enrich, index, and retrieve them.
LangChain	Provide the LLM interface and tool-calling/ composition glue.
LangGraph	Orchestrate intent identification, retrieval, metadata filtering, deterministic tool use, verification, and final response.
MCP	Expose narrow deterministic capabilities: `RunQuery`, `LogFetch`, `RunDiff`, and `CommandBuilder`.
Postgres + S3/MinIO	Retain structured truth and artifacts.
Vector database	Retain semantic representations for retrieval.

How the concepts apply. This is a concrete counterexample to treating all four technologies as interchangeable. LlamaIndex did not authorize tools, LangChain did not become the durable source of benchmark truth, LangGraph did not replace retrieval, and MCP did not decide the workflow. The host/orchestration layer still owned what the model could request and whether a risky action required approval.

Design decisions and trade-offs.

- Using several framework layers added dependency, upgrade, tracing, and debugging complexity. The justification was distinct responsibility, not framework popularity.
- Narrow MCP tools were less flexible than `run_shell` or `execute_sql`, but their schemas and deterministic implementations made validation, auditing, and reproducibility possible.
- `CommandBuilder` used allowlisted templates inherited from the automation platform. This constrained the model, but avoided free-form generation across DPDK crypto's 23+ commands and 10+ variables per command.
- BIOS- or reboot-affecting operations retained a manual gate. That added operator delay but contained a known high-impact risk.

Outcome. The architecture connected grounded evidence to safe operational capabilities, while commands and comparisons remained deterministic. Tool calls were auditable, and CI evaluation covered tool reliability as well as answer quality.

Senior/Staff interview framing.

- **Senior:** trace one request and name the input/output contract at each framework boundary. Explain what happens when retrieval is empty, a tool fails, or verification finds unsupported claims.
- **Staff:** state the selection principle first: "adopt a layer only when it owns a distinct problem." Explain the organizational benefit of reusable tool capabilities and the operational cost of a larger stack, plus the trigger for simplifying or replacing a layer.

Evidence boundary. The project documents an MCP tool boundary but does not describe transport choice, JSON-RPC message traces, capability negotiation, session

handling, or MCP server topology. Do not infer those protocol-level implementation details.

11. Interview questions

1. What is LlamaIndex's center of gravity?
2. Retriever versus query engine?
3. LangChain versus LlamaIndex?
4. LangChain versus LangGraph?
5. Why might direct provider code be better?
6. What does MCP standardize?
7. MCP versus function calling?
8. Host versus client?
9. Tool versus resource versus prompt?
10. Why is an MCP session not authentication or memory?
11. How do you govern hundreds of tools?
12. What must happen before a state-changing tool runs?
13. Why keep framework types out of domain interfaces?
14. What does context augmentation mean?
15. How would you test and roll back a framework upgrade?

12. Exit checklist

- [ ] Classify all five technologies correctly.
- [ ] Explain LlamaIndex documents/nodes/index/retriever/query engine/synthesis.
- [ ] Explain LangChain model/prompt/retriever/tool/chain/state/output/middleware.
- [ ] Explain MCP architecture, lifecycle, capabilities, and JSON-RPC.
- [ ] Design narrow, authorized, approved, auditable tools.
- [ ] Choose the minimum useful stack and state its trade-offs.
- [ ] Test framework adapters and upgrades without coupling domain tests to framework internals.

Source notes

- LangChain Fundamentals
- LlamaIndex End to End
- LangChain End to End
- MCP End to End
- Vanilla RAG and Frameworks
- Capstone Revision Day 2
- DPDK Automation for Network Packet Processing
- DPDK BenchOps Copilot